The bench harness

Ian Ameline · version 1.57 · MIT licence

Three gates, each roughly ten times the cost of the one before it. Run them in order; a cheap failure stops an expensive one.

gate	cost	needs	what it answers
`tools/release_sweep.py --offline`	~1 min	nothing	can this code possibly work
`tools/bench_smoke.py`	12.8 min	both instruments	is the bench and the harness sound
`tools/soak.py --suites formats,plan,arm --skip-vectors v95,v96,v97`	~2.7 h a lap	both instruments	does it hold up over hours, and does the arm still fire

Measured, not estimated: **6.5 s per cell**. Ask the planner for the lap size rather than reciting one — `python3 tools/soakplan.py --iteration 1 --skip-vectors v95,v96,v97` prints it, and it moves whenever a waveform is added. Today that is **40 waveforms at 43 rates each, 1720 cells**, so about **3.1 hours**; all 43 waveforms is 1849 cells and 3.3 h. An offline lap of the same cells is **about 4 seconds** at one capture offset per cell and **19 seconds** at eight, measured on 12 workers.

Figures further down this document that divide by a **1677**-cell lap were measured when the suite held one waveform fewer. They are left as they were recorded: restating a lap size does not change what those runs counted, and rewriting the denominator of an archived result would.

Eight hours is therefore between two and three laps. That is the floor, not a target: the point of the soak is a failure rate per point, and two laps can only ever say "twice" or "once" or "never".

The seeded sweep

`tools/soakplan.py` decides what one iteration tests. Everything comes from the iteration number, so **iteration 129 is reachable without running 128 first**.

Per iteration, four things vary and nothing else does:

- the **order** waveforms load in, so a state leak out of one stops looking like a defect in whichever waveform always follows it
- the **non-standard rate** drawn in each gap between standard rates
- the **wait** before each capture, per cell
- the **vertical placement** of each cell: a target logic swing drawn uniformly from **0.45 V to 8 V**, and a DC offset drawn from what is left inside the rails

The vertical draw, and the one region it will not use

The swing is asked for **in volts**, not as a scale factor. Scaling each vector's own rendered span bottoms out at 1.65 V p-p on the 3.3 V vectors, which left the bottom of the claimed range untested by construction; asking for a span in volts covers 0.45 V to 8 V on every vector that can reach it, and reports the achieved figure per row so a generator ceiling shows up as a ceiling rather than as coverage. A 3.3 V-span vector needs 24.2 Vpp for an 8 V swing and the SDG stops at 20, so its achieved span caps at **6.600 V** — stated, not silently absorbed. The cap is the vector's own codeword range against the amplitude ceiling, so it is per family: at the top of the draw `soakplan.amp_ofst_for` returns AMP 20.000 / OFST 0.000 for `v41`, and

`assert_unclipped` gives the band 0.000…6.600 V, where the LIN vectors reach the full 8.000 V and `v47`, `v48a` and `v48b` reach 8 V by spending part of it below ground. `docs/VECTORS.md` carries the codeword range of every family and the arithmetic.

The offset draw skips the window that would put a single-supply band across ground, and that exclusion is not cosmetic. `sig_levels` decides idle polarity from the levels when no run in the window reaches ten bit times: `lo < −flatfloor and hi > flatfloor` means RS-232, which marks at its *negative* level. So a 0…6.6 V logic waveform shifted until its low is below −1 V and its high above +1 V is, to the app, correctly read as an inverted line — right rate, right format, self-consistent bytes matching nothing, one frame short. Unconstrained, **17.3 % of a lap's cells land in that window and they account for 86.8 % of all offline failures** — enough to put the true rate at ~0.23 % behind a measured 1.79 %, and enough to manufacture a "failures rise with swing" gradient, since a large span is what lets a shifted band clear 1 V on both sides.

Only the straddling window is cut, and this matters: raising the lower bound instead also removes the legitimate wholly-negative region, ~57.5 % of the interval on some vectors, where an offset of −6.5 V decodes while −3.6 V does not. `soakplan.py --selftest` asserts the span range, that the draw reaches both ends of it, that nothing clips, and that nothing straddles ground. `FLATFLOOR` is **read out of `tsp/serial_core.tsp`** rather than copied, so the constraint cannot drift away from the rule it exists to respect.

Fixed: the 22 standard baud rates in `[300, 250000]`, tested on every waveform every iteration. `250000` is `sdec.maxbaud`; below 300 costs 8 s of capture for a rate nothing here speaks.

Frame mode only. Above 165563 Bd `sdec.fs_for_burst` returns nil, so the streaming paths cannot record 172800 and up at all, and only frame reaches 250000.

Why the wait is scaled, not constant

Uniform over **10 byte-times** — which is 100 bit-times — so one continuous draw randomises the byte a capture lands on *and* the bit *and* the phase within the bit, because it is quantised to none of them. Scaled by baud it costs 0.4 min across a whole sweep where a flat 1 s costs 14.7.

It also lands where it is needed. Below about 20 kBd, socket jitter is smaller than a byte time and cannot move the byte offset at all; above it, jitter alone already scatters the phase by tens of bytes.

Why a uniform draw is not the only rule

`sdec.pick_fs` snaps **up** through a coarse ladder, so below 125 kBd samples-per-bit is a sawtooth whose minimum in each step sits at `baud = floor(fs/8)` — 312, 625, 1250 … 80000, 125000. Those are where the decoder is closest to being confidently wrong, and a uniform draw lands on one with probability about zero. A gap containing an edge therefore yields that edge one time in three. Above 125 kBd fs is pinned at 1 MSa/s and samples-per-bit falls smoothly to 4.00, so there is no cliff and uniform is right.

The smoke gate

Four waveforms, then the button matrix.

```
standard rates   v77  the fox, 8N1, 133 B repeating
                 r06  random,  7E1, 256 B non-repeating
drawn rates      v78  the fox, 7E1
                 r00  random,  8N1
```

Paired crosswise on purpose. Each rate family faces both formats and both content classes. Pairing by family instead — fox on the standard ladder, random on the drawn rates — would leave the standard rates never seeing a non-repeating payload, and that is where head alignment and the parity vote get tested.

86 cells in 9.5 min, then 45 presses in 1.9 min. `--plan-spec vector:std|nonstd|all` is what makes the quarter-coverage expressible; a cell's wait is still keyed on its index in the **full** rate list, so a cell reached this way is the same cell the soak runs.

Then two stages of about a minute each. **`levels`** plays one waveform at 5 V, 3.3 V, 1.6 V, 1.0 V, 0.5 V, 0.33 V and 0.25 V of logic swing and checks the `LOGIC` cell, that `THRESH` landed within a quarter of the swing of mid-swing, and the bytes — the published range is 0.33 V to 8 V, and the plan stage drives its own *drawn* amplitude rather than the claimed floor, so nothing else in the gate tests it. **`rates`** runs eight wrong-lock cases from `bench_break`, which is the only stage where the lock CONTRADICTS the wire. `rates` goes last because it stubs `sdec.ua_badfrac` on the instrument, so a teardown that failed cannot quietly change another stage's verdict.

`--no-panel`, `--no-levels` and `--no-rates` each skip a stage, and each writes a **zero** into the receipt for what it skipped, so a partial run cannot later be read as a full one.

The receipt

A pass writes `out/smoke-receipt.json` holding a hash of `tsp/` and `tools/`. `tools/hooks/pre-push` recomputes that hash and refuses a push when either tree has moved, so "I ran the smoke test" becomes a claim about the exact code being pushed rather than about the past. The hash folds in the working tree, not just the index, so an unstaged edit invalidates it too.

`SMOKE_OVERRIDE="reason" git push` proceeds and prints the reason. Docs-only pushes need no receipt.

Changing your mind mid-run

`--hours` and `--laps` are both decided before the first lap, so "make that four laps, not three" would otherwise cost a restart — and a restart throws away the laps already banked, which is the opposite of what one-more-lap means. Two files in the record directory are read before each lap:

```
echo 4 > <record dir>/LAPS      # run exactly this many laps, then stop
touch  <record dir>/STOP        # stop cleanly after the lap in flight
```

Neither interrupts a lap. **A lap is the unit of evidence** — a truncated one is not tallied, so cutting one wastes every minute already spent on it. For the same reason the wall deadline from `--hours` only gates *starting* a lap: one that begins inside the budget always runs to completion.

The release gate, stage by stage

`tools/release_sweep.py` runs every check in one go and writes an auditable record to `out/release/<timestamp>/`: a `REPORT.md`, a `summary.json`, every stage's full output, every front-panel screenshot, and the exact `.tspa` and `MANUAL.pdf` the results describe with their SHA-256s. It exits non-

zero if any gate fails. `--offline` skips every stage needing an instrument and runs in seconds; `--skip name,name` drops named stages, and a skipped gate is reported as *not passed* rather than passed.

The full sweep needs a freshly power-cycled DMM6500. `sdec.start()` builds the display objects and the app refuses a second build in one power cycle, so the sweep checks that up front rather than failing forty minutes in. It also needs the generator loaded with the stimulus waveforms (`bench_matrix.py --upload`), and only one client may hold the DMM's control socket at a time.

Which generator and scope will do

The bench here uses an **SDG2122X**, but nothing in this project needs the top of that line. What it actually asks of a generator, measured from the plan and the driver rather than from a datasheet:

Requirement	Where it comes from
TrueArb at an explicit sample rate, up to **25 MSa/s**	the highest `srate` any plan cell draws. `SDG_MAX_SRATE` is capped at **40 MSa/s** — the family's floor, not this unit's 75 — and `bsdg.select()`, `bench_matrix`, `bench_sweep` and `soakplan --selftest` all refuse a cell above it, so a future rate ladder cannot quietly outgrow an SDG2042X
20 Vpp into high-Z	the largest amplitude the plan draws (`bsdg.maxvpp`)
~41 stored arbitrary waveforms, selected by name	`C1:ARWV NAME,…`, uploaded once by `tools/upload_vectors.py`
Signal bandwidth under ~1 MHz	the fastest stimulus is 250 kBd, so its edges are slow by any generator's standard

The family floor is the 2000X, and an SDG1000X will not do. TrueArb is selected by `SRATE`, and Siglent's programming guide (`docs/SDG_Programming-Guide_PG02-E05C.txt` , the per-command availability table) lists it family by family:

	SDG800	SDG1000	SDG2000X	SDG5000	SDG1000X	SDG6000X/X-E	SDG7000A
`SRATE`	no	no	**yes**	no	**no**	yes	yes
`INTER`	no	no	no	no	no	yes	yes

So the substitution to avoid is the cheap one: an **SDG1000X is not an SDG2000X minus bandwidth**, it has no settable sample rate at all. It stores and plays arbitrary waveforms DDS-only, and DDS **resamples** the stored points — the exact fall-back `SDG.assert_truearb()` checks for on every load, and fatal here rather than merely degrading, because sub-sample edge timing is the quantity being measured. The scope being an SDS**1000X**-E is what makes this easy to get wrong; the two "1000X" families are unrelated.

`INTER` being `no` on the SDG2000X is also worth keeping in view: the interpolation-method control exists only on the 6000X/7000A, so a 2000X in TrueArb emits a plain held staircase and there is nothing to switch off — which is consistent with what the rise measurements above show the generator doing.

Those are series-wide properties of the **SDG2000X** family: the arbitrary-waveform engine, its sample-rate range and the 20 Vpp output are the same across it, and the model number is the *sine* bandwidth — 40 MHz on the SDG2042X, 80 on the 2082X, 120 on the 2122X. Even the bottom of the range has more than a hundred times the bandwidth a 250 kBd edge needs. **So an SDG2042X, the cheapest of the line, is sufficient**, and the extra bandwidth of a 2122X buys this project nothing.

Only the SDG2122X has actually been run. That claim about the family is from the published series specifications, not from a second unit on this bench, so on a different one check exactly two numbers before trusting a soak: that `C1:SRATE?` reports **25 MSa/s** accepted in TrueArb mode, and that **20 Vpp** into high-Z is accepted. Both fail loudly rather than silently — `bsdg.select()` refuses an amplitude or rate outside its bounds, and `bsdg.truearb()` re-reads the mode after setting everything else precisely because a silent fall back to DDS resamples the stored points and destroys the sub-sample edge timing this bench exists to measure.

The two documented hazards are firmware behaviour rather than model behaviour, so expect them on any unit in the family: repeated large waveform uploads wedge the LAN service (`tools/instruments.py`), and `C1:ARWV?` can take longer than 5 s to answer after a selection — 15 of 86 cells in one lap were lost to that before `bsdg.select()` learned to retry the query.

A waveform selection that lands late, and what it costs

`C1:ARWV?` answers with the PREVIOUS waveform's name on about one real switch in ten. A selection is a real switch only where the vector changes — 31 cells of a 1333-cell lap, since the plan sweeps rates within a vector — so a per-switch rate must be divided by 31 and not by the lap. Measured on a full lap: **4 of 39 switches**, all at the first rate in a block, with each one succeeding when the *next* cell sent the same name a fraction of a second later.

Retrying the query does not fix it; re-sending the SELECTION does. `bsdg.arwvtries` = 4 at 0.4 s was not enough, and the record says so in its own words: `ARWV? still names … SER_Hello_8N1_Drift10_x10.bin after 4 tries selecting SER_Jitter20pct_x100`. So `brun.selretry` re-sends the whole selection, guarded by `bsdg.nfail` — a stale name arrives as a *reply*, while a wedged generator raises `nfail`, and retrying a wedge would spend a minute a cell proving what the first attempt proved.

Measured over 8 laps and ~233 real switches: 19 landed late, 18 were repaired, 1 was lost. Before the retry all 19 would have been stimulus-free cells. `brun.nselback` counts them and the run-total row carries it, which is the only thing distinguishing "it stopped happening" from "it is being fixed".

The selection is sent once per waveform, not once per cell

A lap sweeps every rate across one waveform before moving to the next, so **97.7 % of cells used to re-select the waveform that was already playing** — and that is not free in either direction. The `ARWV NAME` write is what makes this generator briefly stop answering, and the `C1:ARWV?` verifying it then blocks on a stall the write itself caused. `bsdg.select()` therefore remembers what `ARWV?` last *confirmed* and sends neither when the name is unchanged.

per 86 cells (2 waveforms × 43 rates)	re-selecting every cell	selection cached
SCPI messages	602 (7.00 per cell)	434 (5.05 per cell)
`ARWV NAME` writes	86	**2**
`ARWV?` blocking queries	86	**2**
`SRATE?` blocking queries	86	86
blocking round trips	**172 (2.00 per cell)**	**88 (1.02 per cell)**

`C1:SRATE?` is still asked on every cell **deliberately**: neither manual says whether `BSWV` resets the sample-rate mode, and a silent fall back to DDS resamples the stored points and corrupts the sub-sample edge timing this bench exists to measure. That is the one failure worth a round trip per cell.

The cache is set only from a *confirmed* reply, never from the command, so it cannot assert a switch that did not land; every failure path, `bsdg.close()` and `bsdg.reset()` all forget it, which is what lets `brun.selretry` re-send rather than re-skip. `bsdg.arwvevery` = 50 forces the verify back periodically so a cache made stale by something outside this module — a front-panel press — can only misattribute a bounded number of cells.

`bsdg.arwvevery = 0` restores the old conversation exactly (measured: 602 messages either way), which makes this its own A/B: one lap of a plan each way, and the wall-time difference is what the skip is worth. `tools/run_bench.py --no-sdg-skip` is the B half. The traffic reduction is measured on the host; the *time* it buys is not, because the cost is the generator's response latency and only the generator can be asked.

Looking at one operating point more than once

`tools/soakplan.py --emit-csv --repeats N` emits each operating point N times in a row, redrawing **only** the wait, so the capture phase changes and nothing else does. Because the repeats are consecutive and every other field matches, `bsdg.select()` sends the generator **nothing at all** for the 2nd..Nth — a repeat costs a capture and a decode instead of a capture, a decode and seven SCPI messages.

The wait is drawn in **byte times**, not milliseconds, which is what makes it mean the same thing at every rate: ten byte-times is 0.4 ms at 250 kBd and 333 ms at 300 Bd. A fixed 0–9 ms window would cover 2250 bit times at the top of the ladder and 2.7 at the bottom.

Repeats are for a **diagnostic** lap, not for the soak. One capture per condition cannot separate "always fails" from "fails at some capture phases", and a coin flip read as a property has already faked a leak here once. A soak looking for new failures wants more operating points instead. `--repeats 1` is the default and emits byte-for-byte what it always did — the first look keeps the original four-element draw key, because MT19937 seeds from the whole array and appending even a zero would change every wait in every plan already on disk.

That one loss is why `selretry` is 5 and not 3. Two further attempts cost nothing on the 1638 cells a lap that re-select what is already playing, nothing on a switch that lands first time, and are spent only where three attempts have already been spent and lost — while a lost switch is a stimulus-free cell counted against the app for something the generator did.

It is not the waveform's size. Over the qualifying lap's 39 switches the 4 that missed average **15 878 B** and the 35 clean ones **44 679 B**; the two largest at **205 200 B — 53x the smallest that missed — both switched cleanly.** So there is nothing to be gained by scaling any wait with the waveform. All misses on record are at the first rate of a block, i.e. the switch itself is the trigger.

The scope is OPTIONAL. No stage of the release gate contacts it — `tools/scope.py` is reached only through `tools/instruments.py`, by hand, when a stimulus needs verifying independently of the app. The whole gate runs on a DMM and a generator.

Where one is used, the oracle here is an SDS1204X-E and `tools/scope.py` is written against the **SDS1000X-E** family rather than that model: the two decode buses, the decode availability table and the 1 GSa/s waveform read are all series properties. What it is asked for is UART decode, two buses, and 1 GSa/s for the tiebreaker read; the model number is again the analog bandwidth, 100 MHz on the **SDS1104X-E** against 200 on the 1204X-E. A 250 kBd bit is 4 µs and the impairments under test are ~2 % of one, so 80 ns is the finest edge displacement that matters — three orders of magnitude inside either model. **An SDS1104X-E is sufficient.**

Both family claims rest on the published series specifications; only the SDG2122X and the SDS1204X-E have been run on this bench. Nothing enforces the scope's limits in code the way `SDG_MAX_SRATE` does, because the scope only ever reads.

Stage	Checks
`lint` `parse`	Lua 5.0.2 incompatibilities; `luac -p` on every module
`vecrefs`	every waveform id named in `tools/` is in `MAP` or declared `RETIRED` — deleting from `MAP` alone leaves references that fail where they are used, not where they are declared
`soakrand`	`mt19937.py` and `mt19937.lua` produce one sequence, checked against CPython's `random`, plus the 5.0.2 scan deciding whether the module can load on the instrument
`unit`	1 194 offline tests — decoder, UI, state machine, file paths, every visible ASCII glyph
`unit-cancel`	one-press recordings, both window sizes, and the flow-control loop with no interaction
`unit-frontrig`	TRIGGER-key and rear-BNC arming *through* `acquire()` — the routing that can drop to free-run while the status row still claims the key is the source
`unit-usblog`	USB log name allocation and the persistent index: exhaustion must refuse rather than loop and hang the panel
`unit-stream`	the streaming arm — both 4915 defences, one press per slice rather than per window
`unit-patterns`	byte-exactness on the hard payloads: edge-density extremes, walking bits and known random data, at the sample rates the panel actually picks
`unit-forcerate`	a forced rate the wire does not carry must **refuse** and name the rate it does carry, with both answers drivable unattended
`unit-judgev`	the harness's three verdicts — too short to read is **inconclusive**, not silently wrong; an alignment survives one bad byte; a loud vector may miss a bounded few. Each with the wrong capture that must still fail, and the offline twin's copy of the allowance *executed* under `lua` against this file's own expression
`unit-ratefit`	the #46 rate-misfit window, pinned per cell and per capture phase against a recorded table — every cell that reproduces must reproduce at every phase, and every cell that does not must be silent at all of them
`unit-octave`	the octave misread — 9600 Bd reported as 19200 `7N1` with the bytes **silently** wrong, which is the decoder's worst failure shape because it is confident rather than refused. Asserts a *fraction of a swept space* rather than one capture: the defect needs the app's real rate-selection path, which takes up to three captures, and it appears only for some capture start points. The sub-multiple gate is also stubbed out and the bench signature must return exactly, so a reproduction too gentle to provoke anything cannot pass as a clean one. Carries **two declared expected failures** (#40 — the same width collapse reached through a *forced* rate, open on purpose); the suite exits 0 with them, and an expected failure that starts *passing* fails the stage, so the declaration cannot outlive the defect
`unit-snapbias`	**a null, and it gates as one.** Jitter alone must not bias the width fit into the neighbouring standard rate: the measured fit error at 10 % and 25 % jitter against the margin `sig_snap` needs, the firm band strictly inside `snaptol`, and the `BAUD` cell agreeing with the text row about the trailing `?`. If a change ever does make plain jitter bias the fit, every rate near the standard ladder is at risk rather than the handful of drawn ones, and these numbers move

Stage	Checks
`unit-trigmodel`	**the trigger subsystem as a machine, and the two arming defects that used to need hardware.** The comparator is *derived* rather than declared: the mock scans the render for the first crossing of `dmm.digitize.analogtrigger.edge.level` in the direction `.slope` names, so a level outside the signal's swing never fires and the wait expires — which is what made the shipped 1.63 V arm (a *previous* capture's midpoint, mid-data on a 0–3.29 V line) pass every offline test. `hw-arm` H, I and M measure what the comparator really holds; this stage is the offline half, where arming at the wrong level has a *consequence*. The 7.3 µs comparator latency is charged **in time**, so the same crossing lands 7 samples late at 1 MS/s and rounds away at 40 kS/s. And the template's blocks are walked on a virtual clock that `delay()` advances, with **readiness and candour as two different times**: the wait goes live 2 ms after `initiate()` and `trigger.model.state()` does not admit it for another 190, both flat in the buffer, the admission moving only with what the instrument was doing before the arm (0.228 s after aborting a model that was itself waiting, 0.164 s after a completed `digitize.read()`). Two gated nulls hold the shape — a buffer a thousand times deeper must reach its wait at the same time, so the refuted per-reading scaling cannot come back; and an arm whose event arrives *inside* the lag must never be reported as waiting at all, which is the mechanism by which a working capture gets declared unarmed. Two assertions fail when their correction is removed from `tsp/` — the comparator armed at the reused midpoint, and `sdec.arm_settle_s` set under the measured admission floor — and a third fails at the old 0.25 s, which clears the slowest admission by 1.1× where the shipped 10 s clears it by 44×
`unit-plansweep`	the offline twin against the soak's own drawn plan, ratcheted on thirteen counters at the configuration each baseline was measured at — iteration, offset count **and skipped set**, because all three change the draw
`unit-soaklog`	the soak's own lap log replayed through the completeness tests, so a lap that recorded nothing cannot read as a lap that found nothing
`unit-benchengine`	the on-instrument soak engine, driven down the branches a run on the box cannot arrange on purpose: a dead generator mid-lap, a waveform file it cannot read, TrueArb falling back to DDS, the plan ending, the key refusing a write — plus the fidelity check that `brun.point()` measures what `bench_point()` measures, the Python extracted and run side by side
`unit-pushplan`	the acknowledged-batch handshake that puts ~280 000 plan rows on the key over the LAN. A protocol, so what is gated is disagreement between the two ends — a batch silently short, an acknowledgement that skips, a closed handle — none of which can be provoked on hardware on purpose
`tspa-current`	the committed `.tspa` is what the current modules build. A stale package installs code no test in this repo has ever run: every harness loads `tsp/*.tsp` over the LAN, so only a Manage Apps install gets the packaged code, and stale code still lints and parses
`stress`	hostile signals: never silently **wrong**, never **raises**
`unit-analog`	the bench cases at the app's own sample rates, swept over sampling phase, jitter, noise and where the window opens
`unit-phasesweep`	every vector × capture start × phase/jitter/noise, sharded across the cores — no raise, no result without a format, no wrong byte among trustworthy calls
`unit-seam`	a capture whose arb loop seam lands late must still be judged, and the narrower trim still required
`unit-sdgguard`	every route by which an out-of-spec waveform could reach the generator refuses it

Stage	Checks
`unit-armsuite`	**the arm suite's own judging, offline.** `hw-arm`'s seven cells run on hardware once a lap; this is the half that can run without instruments, and it arrived with 48 checks and nothing to run them. Each of the three reserve-polarity defects is replayed as the field dict the instrument actually produced and must be *caught*: the frame capture that read its pre-trigger reserve as the line's idle, the recording that inherited an inverted format from priming, and the threshold six millivolts from a rail that made a transmitting device read as silent. The negative controls run in both directions, which is what stops the cells passing vacuously — an expiry that returned at once instead of waiting its Arm Wait, a bipolar line read as idle-HIGH, a locked cell that sampled at the blind arm's rate. And it pins the ROW SHAPE against `soak.py`'s own regex and its `ok` / `BAD` vocabulary, because the seven point names contain spaces: a failing arm cell that the soak's parser drops is a defect reported as a clean lap
`unit-soakplan`	the plan draw: pure in the iteration, the skip keys the shuffle, and every cell inside both the rails and the generator's `|OFST| + AMP/2 <= 10 V` envelope
`unit-sdgreply`	the SDG reply parser against known replies, including a DDS mode and an absent key — the probe that reads them touches the instrument, this does not
`unit-loremgate`	the harness's own long-payload verdict: every clean run validated, flag count bounded
`tolerance`	recomputes the envelope table printed in the manual
`package archive`	rebuilds the `.tspa`, then builds both screens *from the archive* against a mock front end
`manual`	rebuilds every shipped PDF: `README.pdf`, `MANUAL.pdf`, `REFERENCE.pdf`, `BENCH.pdf`
`version`	one release version in the manifest, the `.tspa`, every shipped `.md`, every shipped PDF and the git tag. **Read out of the PDFs with `pdftotext`, not inferred from their markdown** — and the tag is checked in both directions: none may sort above the declared version, and this version's own tag must point at a tree that declares it
`stagedoc`	every stage `release_sweep.py` defines has a row in this table — the published inventory checked against the code rather than trusted
`hw-matrix`	through the app's own Capture button: six frame formats, the standard rate ladder, a 1 kB non-repeating payload, **seven logic swings from 5 V down to 0.25 V**, twelve DC offsets
`hw-payloads`	fourteen payloads covering every byte value 0–255, two of them also at 115 200 and 250 000
`hw-odd-rates`	nineteen **non-standard** baud rates — 900, 1500, 3600, 8123, 29127, 104857 …
`hw-panel`	every button in every state, including a one-press recording. Six checks per press: the handler does not raise, logs no instrument event, returns inside its latency budget, reports what it did, changed the state it was supposed to, and **the panel actually shows it** — grabbed before and after each press and differenced by region
`soakrand-dmm`	the third leg: the instrument's **own** Lua 5.0.2 must produce the same words, floats, rejected draws and permutation
`hw-plan`	the seeded sweep on the bench: every waveform across all 43 rates in a seeded order, with a seeded wait, amplitude and DC offset drawn per cell. `--plan-vectors` cuts it to a subset for a lap that must finish in minutes. The vertical draw asks for a **target swing in volts, 0.45 V to 8 V**, and places the offset wherever keeps both levels inside ±9.5 V *and* does not put a single-supply band across ground. Every row records the swing it drove and the `S/N` the app reported

Stage	Checks
`hw-break`	degenerate signals and contradictory settings — no signal, DC only, all-`0x00` / `0xFF` / `0x55`, a break, 60 mV of swing, 19 Vpp, rates past the ceiling, six wrong forced settings. A refusal with a reason passes; confident garbage does not
`hw-arm`	**arming from silence**, which is the one case no offline test can reach — what makes it work is the comparator, and the mock reproduces the program shape around it rather than the comparator itself. Fifteen cases on a line held quiet by switching the generator's output off. **A–G are the feature's own claims**: the default configuration (rate automatic) must *wait* rather than record the silence, a device that starts three seconds into the wait must be caught **from its first byte** with the pre-trigger reserve ahead of it, a line idling **high** at 3.3 V must arm one `Arm At` below idle and on the falling edge, an expired arm must name both settings and the level it was watching, and an 8 kB **recording** must arm the waiting template instead of the one that fires immediately. **H–N are the hostile half**: both settings at their limits (0.33 V must not fire on the digitizer's own noise; 6 V must be *adopted* by the firmware, which only a readback can establish, because a refused attribute write returns success and merely files an event), a 2 s wait against a device that starts too late, a line idling at **–5 V** where the start bit travels up and a wrong sign cannot fire at all, the guessed arm level not outliving its own capture, blender slot 3 really holding `EVENT_DISPLAY` so the panel's promise of a TRIGGER-key escape is backed by instrument state, a **57 600 Bd** device caught blind — which 200 kS/s cannot resolve, and which a slack gate published as 38 400 — and the **32 kB** recording, whose buffer is the one the settle budget has to cover — the sweep below ran to 2 800 000 readings and the `ck_bufmax` ceiling has since risen to 4 076 503, so the shipped buffer sits 46 % above the measured range, on a null that held flat across a 133-fold span of it. Reaching the wait block takes about **0.19 s** and does not depend on the buffer at all — flat from 21 100 to 2 800 000 readings, with `digitize.count`, `samplerate`, template `position` and the poll cadence all nulls — but the worst case is **0.228 s** after aborting a model that was itself sitting in `STATE_WAITING`. So the 0.25 s this feature shipped with stood ten per cent above the slowest arm the instrument produces, and case F duly armed once and failed the next three runs

This table must list every stage `release_sweep.py` defines, or the published inventory of what a release passed is incomplete. **The `stagedoc` stage runs this check**, and it is printed here so the reader can run it too:

```
import re
code = set(re.findall(r"Stage\(\s*'([^']+)'", open('tools/release_sweep.py').read()))
doc  = {n for l in open('docs/BENCH.md') if l.startswith('| `')
          for n in re.findall(r'`([a-z0-9-]+)`', l.split('|')[1])}
assert not (code - doc), sorted(code - doc)
```

Reproducing a failure

A failing cell prints a `REPRO` line carrying everything needed:

```
REPRO iteration 1 vector v48a rate 2400 Bd (std) srate 24000 spb 10 wait 38.5438 ms
      measured-head - nf 2 fmt 8N1 want 8N1
```

Replay it offline, where there is no analogue path and no race:

```
python3 tools/soakplan.py --emit-lua --iteration 1 > /tmp/p.lua
lua tools/sweep_plan.lua --plan /tmp/p.lua --cell v48a:2400
```

Fails offline too → logic, deterministic, debuggable. **Passes offline** → the difference is the hardware: the DAC, the cable, the digitiser, or the capture phase that the seeded wait can only perturb.

What the seed does and does not reproduce

The seed reproduces the **schedule** — which rates, in which order, with what commanded wait. On hardware it cannot reproduce the realised capture **phase**, because that is a race the wait only disturbs. This is why a failure record carries the **measured** head alignment: that is what converts a hardware failure into an exact offline replay. Offline the phase is set explicitly, so it is exact by construction.

The offline twin

`tools/sweep_plan.lua` replays a plan cell for cell with no instrument: it reads the `.bin` the generator actually holds, converts codewords to volts at the amplitude the file was encoded for, resamples to the rate the app would pick, starts at the phase the seeded wait produces, and decodes.

Interpolation is **linear** between arb points, not a step, and the reason is **empirical agreement with the bench, not a measurement of the generator**. A staircase is what the part argues for — TrueArb, with the AD9122's interpolation half-bands bypassed — and it was tried: at iteration 1 over 8 capture phases it agrees with the bench on the same 26 cells as linear and invents fifteen more failures the bench does not have. `--hold` runs it. Linear is kept because it fits, and that is the whole of the case for it.

The generator holds, and the measurement that shows it is rate-independent. `C1:PAVA? RISE` on the same waveform at two arb rates, twenty times apart, with only `SRATE` changed:

arb rate	arb period `T`	measured RISE	hold predicts	linear predicts
125 kSa/s	8.000 µs	**39.5 ns**	8.000 µs	12.80 µs
2.5 MSa/s	0.400 µs	**40.5 ns**	0.400 µs	0.640 µs

Rise does not scale with the arb period at all — 39.5 against 40.5 ns across a 20× change. Linear reconstruction would draw a continuous ramp lasting `2T`, so its 10–90 % time would scale with `T` and differ 20-fold between these rows. It does not differ. What is being measured is the generator's own analog output bandwidth, ~40 ns for a 3.3 V step, which is the signature of a **step** — the DAC holds and the output amplifier slews between held values. This is the discriminating experiment: the absolute number is an instrument property, but its **invariance across rate** is decidable and it decides for the hold.

An earlier `RISE = FALL = 8.02 µs` reading at 125 kSa/s was documented here as proving LINEAR. The reasoning attached to it was wrong twice over: "a zero-order hold would step in nanoseconds" is true only for a pure step, and these vectors are not pure steps — `GEN_RENDER`'s default `rise` is 1.5 samples, so a shipped edge carries one intermediate codeword between the rails, 814 of 814 transitions in `v77.bin`.

The staircase is directly resolvable, and it accounts for that reading. One `v94` edge captured at 1 GSa/s, 50 ns/div, 16-way average, arb at 2.5 MSa/s. The screen shows two fast rises with a flat shelf between them, and the shelf is the intermediate codeword being held:

quantity	measured	
low rail	–0.004 V	
intermediate plateau, dead flat	**0.7527 V**	**46.49 % of swing**

quantity	measured	
high rail	1.6231 V	swing 1.627 V
shelf length, 50 % to 50 % of the two sub-rises	**399.8 ns**	commanded `T` is 400.0 ns, –0.04 %
each sub-rise, 10–90 %	**35.3 and 34.3 ns**	the amplifier slew

`v94.bin` holds exactly four codewords, `0, 10092, 11534, 21626`, and **all 2816 of its intermediate runs are one sample long**, at 46.67 % and 53.33 % of the rail-to-rail span. The plateau is codeword 10092, at 46.49 % against 46.67 % predicted, and it is held for one arb period to 0.04 %. So the DAC holds and the amplifier slews between held values — measured on the waveform, not inferred from a summary statistic.

Which number `PAVA? RISE` returns depends on where its histogram puts `BASE`. On this capture it returned `BASE = 0.7563 V` — **the plateau**, not the low rail, whose `MIN` was –0.013 V — so its `RISE = 35.13 ns` timed only the second sub-rise. With the true rails as `TOP` / `BASE`, 10–90 % instead spans the whole staircase: **420.7 ns, which is `T` + 20.9 ns**, and that 20.9 ns slew term is fixed in time rather than a fraction of `T`. At 125 kSa/s it therefore predicts **8.021 µs**, against the **8.020 µs** recorded. The 40 ns readings and the 8.02 µs reading are one model measured two ways, and the rate-invariance above remains the argument that decides it.

`v47`'s spikes do not discriminate either way and should not be cited as if they do: they are **two codewords wide and flat-topped** (`10813, 10813, 20643, 20643, 10813, 10813`), not the one-sample impulse they were described as, and a symmetric trapezoid and a rectangle have the **same width at half height**, 16 µs. The "~16 µs triangle with a single-point apex" came from digitising at 10 kSa/s, where a 16 µs feature is sub-sample and one extreme sample per spike is a sampling artifact rather than a shape.

So `sweep_plan`'s linear default rests on **fit**, the physics says **hold**, and the two disagree. Linear is kept there because it is what agrees with the bench and because changing it costs a soak to re-baseline — not because the generator interpolates. It does not.

And a ground-truth capture now exists, which is the first time the offline stimulus has been compared against real samples rather than against a pass/fail verdict. 20 000 samples of `v94` at 250 kBd, arb 2.5 MSa/s, 20 Vpp, digitised at 1 MS/s with the 1 µs aperture, run through the app's own `sig_levels` beside both mock variants:

	min/max	app `lo` / `hi`	swing	family
hardware	–0.0159 … 6.6186	–0.0080 … 6.6101	6.6181 V	6.6 Vpp
mock, front end off	0.0000 … 6.5999	0.0000 … 6.5999	6.5999 V	6.6 Vpp
mock, front end on	–0.0000 … 6.5999	0.0007 … 6.5993	6.5986 V	6.6 Vpp

All three agree to within 0.02 V, so at this operating point the mock's **levels** are faithful with or without a front-end model, and the level metric is not what separates them.

The capture's real use is as a metric. Scoring a source model by the judge's fail count is a downstream proxy that can improve for the wrong reason; the RMS residual against real samples cannot. Swing 6.6345 V:

source model	residual	% of swing
zero-order hold — the shipped default	0.51636 V	7.78 %
the 440 kHz + 1 µs front end, then ZOH	0.31189 V	4.70 %

source model	residual	% of swing
the front end, then linear	0.19374 V	2.92 %
linear interpolation	0.09004 V	1.36 %
the aperture as an **integral** of the held staircase	0.01409 V	**0.21 %**
the capture's own noise + quantisation floor	—	~0.14 %

So the faithful model is an **integral**, not a point sample: the generator holds, and the DMM integrates over its aperture. Linear interpolation is a cheap approximation to that average, which is why it beats a hold by 5.7× and still loses by 6.4×; and the arb-domain front end **double-counts**, filtering and then resampling, which is why adding it makes linear worse.

It is implemented and it is the default, as `SRC.aperture` in `tools/gen_serial.lua`, at a width fixed in *time* of 0.85 µs. Against real captured samples it takes the residual from 5.62 % of swing to 1.07 %.

The width is only identifiable at some operating points, which is what made it look untransferable. The count of distinct sub-arb-sample phases the capture visits is the denominator of `STEP = arb/dmm` in lowest terms: at a denominator of 1 the samples carry no information about the width at all, at 2 the fit is observationally **degenerate** — a width of 1.5797 µs reproduces data generated at 2.815 µs to 2 µV — and at 5 it recovers exactly. Earlier fits of 1.126 µs and 1.388 µs came from degenerate points, so they were not two measurements of a drifting width; they were two arbitrary members of a flat valley.

Three things the same captures retired as candidates: **noise** is 1.92 mV sd at held rails, 0.029 % of swing and 2 % of the linear residual; the **arb-to-DMM sample ratio** fits to 2.500000, +0.0 ppm, so there is no clock drift to model; and **resolution is ~11 bits, not 16** — 692 distinct values in 20 000 samples, ~9.6 mV apart where the 10 V range's LSB is 0.305 mV. And **99.7 % of the squared error sits within two samples of an edge** while held levels are already at the floor, so edge shape is the entire remaining gap — which is exactly the quantity `sig_bittime` fits.

`tools/gen_serial.lua` carries five source models, and `tools/offline_bench.lua` prints which are active on every run's `stimulus:` line rather than leaving it to be inferred:

model	`SRC` flag	default	why
aperture integral of the held staircase, 0.85 µs	`SRC.aperture`	**on**	the residual above: 5.62 % → 1.07 % of swing
the rate the clock can synthesise, `66e6/ceil(66e6/fs)`	`SRC.truefs`	**on**	the requested rate manufactures round samples-per-bit the bench never delivers
sub-sample capture origin	`SRC.fracorigin`	**on**	two independent clocks never align on a sample
linear interpolation between arb points	`SRC.interp`	off	under-predicts, and is a cheap approximation to the integral rather than something to compose with it
the 440 kHz pole + 1 µs boxcar in the arb domain	`SRC.frontend`	off	double-counts the aperture

Only the aperture is carried by a **measurement**; `truefs` and `fracorigin` are rate-null and on because they are physically true. `--aperture` with either `--interp` or `--frontend` is **refused** rather than silently ignored: `SRC_val` returns from the aperture branch first, so the other model would be inert while the header claimed it had been applied.

Never set an offline rate beside a bench rate without matching the envelope

`tools/soak_offline_bench.py` defaults to `--stress-envelope 0.508`, which inflates the commanded offset on every second cell until `|OFST| + AMP/2` exceeds 10 V so that `GEN_ENVELOPE`'s recentring law is exercised. That reproduces a condition the archive really ran. **It is not the condition a current hardware lap runs**: `soakplan` refuses any pair past `SDG_ENV_V`, so a modern lap has **0 cells outside the envelope** — checked on the record's own `amp_vpp` / `ofst_v` columns, maximum exactly 10.0000 V. A recentred band straddles ground, `sig_levels` reads it as RS-232 and inverts, so the stressed arm is a *different experiment* rather than a harsher one. Within one stressed run the two halves score 0.911 % inside the envelope against 2.054 % outside it, a factor of 2.26.

Matched (`--stress-envelope 0`), the current defaults reproduce the bench:

arm	laps	cells	BAD rate
hardware	11	18 447	**1.1872 %** (219 BAD)
offline, shipped defaults	265	444 405	**1.1348 %** — 0.96× hardware, 0.67 σ from parity
offline, all five models off	336	563 472	1.7760 % — 1.50×

The aggregate agreeing is partly cancellation, so read the per-vector table too. Offline runs `v90` / `v94` / `j10` / `v63` about 1.3× hot and adds ~1.5 failures a lap on random vectors the bench never fails, while scoring zero on `v48a` / `v48b`.

Zero on those two is not blindness, and the refusal counts are what say so. Both are drift vectors, and both sides decline nearly every cell at the acquisition gate with the same reason, `acq: no clear logic levels (8–9 % of samples near them)` — offline **100 %**, hardware **92–98 %**. Offline is the harsher side; the hardware failures are the ~8 % of drift cells that stay just readable enough to answer wrongly, and a refusal on a `loud` vector is not a failure. Only `v48b`, `v48a` and `v47` refuse at all in a hardware lap (168, 158 and 32 cells of a 1677-cell lap); nothing else does.

`v47` refusals are an independent check on the aperture, on a quantity nothing was fitted to — hardware 8.0 a lap, shipped defaults 9.1 (1.14×), all models off 14.9 (1.86×). A 0.85 µs integrating window attenuates a spike two codewords wide, so the mock stops inventing refusals the bench does not have.

The plan the twin replays is the plan the bench played, which needs the lap's own `--skip-vectors` list. `soakplan` applies the skip **before** the shuffle, so dropping two waveforms moves every remaining vector's index — and that index keys the amplitude, the offset *and* the wait for every cell. Every hardware lap skips `v95` and `v96`, so a twin lap emitted without them drove a different waveform in **all 1677 cells**: `v94` at 300 Bd was 20.000 Vpp at −8.220 V on the bench and 8.595 Vpp at +2.347 V offline. `tools/plan_sweep.py` therefore defaults `--skip-vectors` to the soak's own `v95,v96`, the emitted plan carries its `skipped` set and `sweep_plan.lua` prints it, and the pairing is checkable against a finished lap:

```
python3 tools/soakplan.py --iteration 1 --skip-vectors v95,v96,v97 \
        --check-log out/soak/<dir>/lap0001-FAILED.log
```

which reads the emitted plan and compares every cell against the `N Vpp gen, ofst M` the log records. Without the skip, all 1677 cells of iteration 1 disagree; with it, none do.

The digitiser is modelled, not assumed. Two settings the app pins are fixed in *time*, so their effect in samples changes with the rate, and both are applied in the arb time base before decimation: the 1 µs aperture (`sdec.dig.aperture`) and one pole at the 10 V range's 440 kHz digitize bandwidth. `--raw`

disables them. The sample rate is the one the clock can synthesise, `66e6/ceil(66e6/fs)` — so 80000 Bd at a listed 640 kS/s is 7.93 samples/bit here as on the bench, not a round 8.000.

`badcells` is a union over capture phases and must never be set beside a bench lap's failing-cell count. With `--offsets 8` a cell is called bad if any *one* of eight phases failed; the bench takes one capture and asks once. Measured against soak lap 1: 65 cells fail here and passed on the bench, and not one of them fails at all eight phases — 41 fail at one of eight, 20 at two, none past five. Per *capture* this harness fails 1.09 % against the bench's 2.80 %, so it is not harsher; the union is. `badall` is the intersection — the cells no capture placement saves — and it is the cell figure the two harnesses can be compared on.

What it is not. It is not the app. `bench_matrix` presses the real Capture button and reads the real panel; this calls `sdec` directly, so it never exercises the two-pass probe, autolock, or anything the operator sees — and it never exercises waveform selection, which is why a selection fault shows up on hardware and passes here. In particular the bench digitises at `pick_fs(` the baud its own **probe** measured `)` and this digitises at `pick_fs(` the baud that was **commanded** `)`; on soak lap 1 those disagreed on 26 of 1385 cells, 1.9 %. It also judges more simply: the bytes must be a cyclic substring of the payload — save for the `loud` mismatch allowance below, which is `bench_uart`'s own number — where `bench_uart.judge_payload_v` also weighs coverage floors and head damage.

And it is laxer than the bench in two places, which is the asymmetry that is easy to miss because it runs the other way. The bench fails an inconclusive cell on an `exact` vector where this counts it as unjudgeable — which is why `skip` is ratcheted, so points cannot quietly move out of `bad` into it. And the bench checks the reported **format**, which this does not.

A misreported rate is not one of them. A cell whose bytes are right and whose reported baud is outside `snaptol` fails here, counted as `label` and classified by route, because `bench_matrix` ends its verdict with `good = good and not baudbad` — *after* its `loud` rule, so a waveform's licence to fail on content is not a licence to be wrong about the number on the panel.

"Bytes are right" means inside the `loud` budget, not payload-exact, so a capture that spends its whole budget still counts as `label`. That is where the offline sweep's `label` of 610 and the payload-intact figure of 609 in the manual part company: one `v48a` capture at 149 527 Bd lands on 6 mismatches against a budget of 6. `label` is a ratchet baseline in `tools/plan_sweep.py`, carried beside `bad` and `rate` in every configuration, so know which definition a number came from before correcting either.

Thirteen of the bench's own iteration-1 failures are BAUD rows on `loud` vectors, so a check conditioned on the payload verdict passing would see none of them.

Its head bound is the app's own, not a constant. It trims `max(ua_edge_frames, ua_head_bad)` bytes and then tries a bounded run of further shifts, counting any shift it actually needed as `bleed` rather than absorbing it. A fixed trim cannot work: `ua_head_bad` reads 34 on `v77` at 19200 Bd, so a 12-byte constant failed byte-exact decodes at all 43 rates and manufactured 3377 failures across 20 laps.

What it is worth, measured over a whole run. A 172.3 min offline soak put 2818 plan runs of 1176 points through the shipping tree — 3 313 968 decodes, **321 a second** — against a hardware lap's 1683 cells in 2.9 h, which is 0.16 a second. That is **1989×**: those 2.9 offline hours are **238 days** of soak decodes, about 83 days an hour.

Take that rate from a completed run, not from the first few minutes. The same figure measured over the opening 11.7 min came out 548 a second — 71 % high — because the short suites cycle first and the seeded sweeps that dominate the wall clock had barely started.

And it is decode volume, not coverage. The paths above are covered 0× however long it runs — the two-pass probe, autolock, the panel, waveform selection — and that, not volume, is the argument for running both.

The `rstdC` zero that used to stand here was this harness not looking. Lap 1 of a hardware soak put 15 distinct rate misreports on the board, of which 3 — `37422→38400`, `149527→153600`, `187477→192000` — were the `rstdC` shape at 1.024–1.027×, against **0 in 141 040 decodes** offline. But `judge()` compared bytes and returned ok for a cyclic-exact capture, so `classify()` — the only place a rate is read — was never reached, and a cluster-C misreport *with right bytes* was uncountable by construction. With the rate checked wherever bytes came back, 130 plan draws give **4767 `rstdC` in 1 788 800 decodes, 0.267 %**, of which **3398 are the bytes-right class** that scored the zero, 0.190 % — against that lap's 0.18 %. Capture phase is not what changed it: the same draws at one offset, the historical setting, give **591 and 421 in 223 600** — 0.264 % and 0.188 %.

What a waveform is entitled to do

Three classes, in `soakplan.VECTOR_EXPECT`:

- **`exact`** — must decode byte-exact at every rate. Anything else is a defect. It does allow a few frames the app itself flagged — `max(2, 2 %` of the body`)` — and judges 95 % of what reached it.
- **`loud`** — may fail, and **must not be silently wrong**. Declining, or failing with flags raised, is a pass. Confident bytes that are not the payload fail for these too.
- **`nodrop`** — **stricter than `exact`**, and the only class that forbids a *declined* byte. It withdraws both of `exact`'s allowances, because on a waveform whose whole subject is a long idle run a dropped frame is the defect being looked for, and tolerating two of them would pass exactly that. The six gap vectors carry it. Two limits do the work and they catch different positions: a declined frame in the body's interior fails the flag count, and one at the body's first or last frame — which that count exempts — fails the 100 % coverage demand. A frame clipped at the capture's start is neither, being inside the head the judge skips before any budget is consulted.

The offline twin does not implement `nodrop`, and is therefore more permissive than the bench on exactly the six vectors that class exists for. `sweep_plan.lua`'s `judge()` takes a single `loud` boolean, so a gap vector is judged offline as though it were `exact` — a declined frame spends `exact`'s `max(2, 2 %` of the body`)` allowance and passes, where the hardware judge fails it. The consequence is worth stating plainly: **a clean offline run is not evidence about the gap vectors**, and the defect they were built to catch can only be seen on an instrument.

Left as a divergence rather than closed, for one reason: teaching the twin `nodrop` changes its verdicts, and `tools/plan_sweep.py`'s baselines already need re-measuring against the grown vector set. Doing both in one step would make neither attributable. The two belong in the same decision, and that decision is about which vectors the ratcheted configuration means.

A `loud` vector may also miss a bounded number of bytes, and the bench says how many. In 8N1 a data sample that crosses into the neighbouring cell yields a wrong byte with framing intact and no parity to catch it, so nothing is flagged and a jitter vector cannot be held to zero — `bench_uart` measured `j20` losing 1–5 bytes in 256 at 172800–240959 Bd and allows `max(2, 3 %` of the judged body`)`. The offline twin allows the same — sized inside its shift loop, from the body it is actually comparing, so a shifted match cannot spend an allowance its longer unshifted body earned — and `tools/test_judge_v.py` *executes* the twin's `loud_budget` under `lua` and compares it with `bench_uart`'s expression at ten body sizes, so

the copy cannot drift in its constants or in its arithmetic. Holding a `loud` vector to byte-exactness where the bench allows 3 % accounted for **11 of the 73 cells** the twin failed and the bench passed on soak lap 1 — seven `j20` cells the bench logged as "6 mismatched of 6 allowed" and the like, and four `v47`. Every byte the allowance forgives is counted as `loudmiss` and ratcheted, for the same reason `loudquiet` is: a tolerance nobody counts is one nobody can bound. An `exact` vector still gets zero.

The app's own flag budget is applied too, and it is `bench_uart`'s: a capture whose bytes are right but which flagged more interior frames than `max(2, 2 %` of the body `)` fails as `flags`, counted on the same trimmed body the bench counts on. It measures **zero** at every ratcheted configuration and at iterations 1 to 5, which is the finding rather than a disappointment: the judge now applies the bench's rule, so the remaining disagreement on `v46` at 80000 Bd — five interior frames flagged on the bench, 203 bytes cyclic-exact and nothing flagged here — sits in the signal chain and not in the judge.

A decline is a pass, and it is still counted. `sweep_plan.lua` tallies these as `loudquiet` — 5 a lap at one capture offset, 54 at eight, skipping `v95,v96` — and `plan_sweep.py` ratchets that count at every measured configuration. Both halves matter: calling a correct refusal a failure is a harness inventing defects, measured at about 80 a lap when it was tried, while leaving it uncounted means a regression that suppressed *every* byte on *every* `loud` vector would move no counter at all and read as a completely unchanged run. The ratchet is deliberately one-directional: loud vectors starting to decode is not a defect, so only a rise is gated.

A RAISE IS NOT A DECLINE, and the licence to fail must not cover it. A decode that threw arrives with no result, which is indistinguishable from an honest refusal unless the raise flag itself is tested — so granting the `loud` pass without checking it leaves `raised` at zero while the decoder is throwing, and `raised` is the one counter that gates unconditionally at every capture placement.

`loud`: `v47` (spikes stacking to 9.3 V), `v48a` and `v48b` (drift), `j20` (20 % jitter), `v61`–`v63` (LIN carries framing violations no UART frame can contain).

`v48a` is the clearest case. Its 0.6 V drift is inside tolerance **at its native rate**; swept two decades away the drift-to-window ratio changes, the two logic levels stop being the two densest amplitudes, and the app reports `baseline unstable — only 8 % of samples sit at a logic level` and declines. Declining is the correct answer to an unreadable signal, so it counts as one.

Ambiguous framing, and why the oracle accepts two answers

`v90`, `v92` and `v94` are blocks of `00/FF/55/AA` and walking bits. In every one of those byte values **bit 7 carries exactly the parity a 7-bit reading would require**, so the wire is a valid 8N1 frame *and* a valid 7E1/7O1 frame, and nothing in the signal separates them. `v92`'s `exp_hex` is its `.txt` with bit 7 stripped — `80 → 00`, `FE → 7E`.

Which reading the app votes for shifts with the rate, so `v94` reads 8N1 at some rates and 7E1 at others, correctly both times. The judge therefore accepts **either** reading, and the format check stands down where there is more than one. Demanding either single answer fails a correct decode at every rate that chose the other.

The rate ceiling, and the quarter of every lap below eight samples a bit

The digitize ladder stops at 1 MS/s. `sdec.pick_fs(baud, 8)` asks for eight samples a bit and returns the lowest listed rate that gives it, or the ceiling when none does — so **every rate above 125 000 Bd runs below the target, and no software choice changes that.** The plan's top eleven rates across 31 vectors are **341 of a 1333-cell lap, 26 %**, and the floor is **4.00 sa/bit at 250 000 Bd**:

baud	sa/bit at 1 MS/s
128 000	7.81
153 600	6.51
192 000	5.21
250 000	4.00

What that costs, measured over 204 freshly drawn laps at eight capture phases — 2.18 M judged points. The `nobytes` class, meaning nothing decoded at all on a vector that should decode, occurs on **6 laps and 7 points: 3.2 parts per million.** Every single one is **`v90` or `v94` and no other vector**, which is what `tools/plan_sweep.py`'s ratchet comment already claimed and this confirms against fresh draws. Four reproduced exactly: three at 153 600 Bd / 6.51 sa/bit and one at 161 259 Bd / 6.20. All four report **`read nil Bd`** — no bit time measured at all, rather than a wrong one.

It is **phase-dependent, not systematic**: `badall`, the count of cells failing at every one of the eight capture phases, is 0 on those laps. The same cell decodes at the other seven placements, so what fails is a particular alignment of a 20 000-sample window against a block payload at 6.5 samples a bit.

The rate is necessary and NOT sufficient, and the rest is not characterised

It would be comfortable to stop at "the instrument samples at 1 MS/s", and it would be wrong. **341 cells a lap run below eight samples a bit, across all 31 vectors, and only two of them ever return nothing.** Every other vector decodes at 6.51 sa/bit — including at 4.00 — so the sample rate is necessary and not sufficient, and something about `v90` and `v94` is the rest of it.

`v92` is the control, and it settles which property matters. All three of `v90`, `v92` and `v94` are "blocks of `00/FF/55/AA` and walking bits" built on the same parity construction, so any explanation resting on that construction has to explain why only two fail. The payloads separate them on one axis:

vector	payload	distinct byte values	longest run of one byte	ever returns nothing
`v90`	256 B	4	**64**	yes
`v94`	512 B	4	**128**	yes
`v92`	240 B	16	**1**	**never**

So the failing vectors are exactly the ones carrying **long constant-byte runs**, and the one that never fails is the one whose bytes never repeat. Inside a 64- or 128-byte run of `00` the wire is nine bit-times low and one high, over and over: the bit-time estimator is handed **two run lengths and no others**, and at 6.5 samples a bit those are ~6.5 and ~59 samples. A 20 000-sample window can land wholly inside such a run, which is also why the failure is phase-dependent.

That is a narrowed hypothesis, not a mechanism. `read nil Bd` says a bit time was never settled on, not why. It is **not** the known second fixed point in `sig_fit` — that one needs `frac(sa/bit) > 0.6` and these are 6.51 and 6.20, fractions 0.51 and 0.20 — so it is a different degeneracy and the entry condition is unknown. No minimal case has been reduced.

IT IS THE RATE MEASUREMENT AND NOTHING ELSE, and locking the rate proves it. The failing capture was re-run at all eight phases with `sdec.force_baud` set to the commanded 153 600, which is what a user typing into `Options ▸ Baud Rate` does:

	8 phases	`nobytes`	per-capture failure
unlocked	7 ok, **1 BAD**	1	**12.50 %**
locked to 153 600	**8 ok, 0 BAD**	0	**0.00 %**

So **6.51 samples a bit is enough to decode with** — the same samples, the same phase, byte-exact once the rate is supplied. What 6.51 samples a bit is not enough for is *measuring* a bit time from a payload that offers only two run lengths. That narrows the class to the rate-detection step and rules out framing, level detection and byte extraction, all of which work fine at 4–6 sa/bit given a rate.

And the decoder is deliberately NOT being changed for it. The bit-time estimator currently reads 2.7 M offline points with zero raises; re-tuning it to chase a 3.2 ppm case, at rates above 115 200, on two synthetic vectors, would put that at risk. The remedy is documented in `docs/MANUAL.md` as the third known failure of automatic rate detection, alongside the two that were already there — and it is the same remedy as both of those: supply the rate.

How much of this a real user can reach

Narrow, and the rate half is the reason: **every standard baud rate up to and including 115 200 is above the eight-sample target**, because 115 200 × 8 = 921 600 fits under the ceiling.

baud	sa/bit	
9 600 – 76 800	8.33	fine
115 200	**8.68**	**fine — the commonest fast rate is not affected**
128 000	7.81	below target
153 600	**6.51**	the band where this is observed
230 400	4.34	below target
250 000	4.00	below target

So reaching it needs a line **above 115 200** — 153 600, 230 400 and 250 000 being the real ones — *and* a payload carrying a long run of one repeated byte, *and* an unlucky window placement.

The payload half is not exotic, though. These two vectors are synthetic, but the property that trips them is not: erased flash reads back as a solid run of `FF`, padding and protocol fill are runs of `00`, and a memory dump over a serial link is mostly one or the other. A `00`-padded frame at 230 400 baud is an ordinary thing and sits squarely in the affected region.

An idle line is *not* an instance of this — idle is continuous mark with no framing at all, which the decoder handles separately. It takes a long run of framed bytes that all happen to be equal.

These cells stay counted as failures, deliberately. The tempting fix is to add `v90` / `v94` at the top rates to `VECTOR_EXPECT` as `loud`, which makes refusing there a documented pass and takes the number to zero. That would be hiding it: `loud` is for waveforms *built* to break something, and these two are clean traffic the app should read. Reclassifying them would also suppress any genuine defect at exactly the rates and the vectors most likely to expose one. Four visible, bounded, explained cells over a hundred laps are worth more than a clean column.

What it predicts for a hardware lap. A bench cell takes ONE capture, not eight, so the offline rate translates to **0.034 cells a lap — about 4 cells over 111 laps.** Those are counted as **UNEXPECTED**. So a long run finishing with an `UNEXPECTED` of three to five on `v90` / `v94` near the top of the rate range is **this**

class, which is understood as far as the paragraph above and no further — not a generator fault, and not a reason to discard the run. Anywhere else, or many more than that, is neither.

The random numbers

`tools/mt19937.py` and `tools/mt19937.lua` must produce one sequence, because the hardware sweep is driven from Python and the offline suites draw in Lua. `tools/test_soakrand.py` holds three legs together:

```
CPython random  ->  mt19937.py  ->  host Lua  ->  the instrument's own Lua 5.0.2
```

CPython's `random` **is** this algorithm, so it is an independent implementation rather than a pasted table of numbers. The Lua side is arithmetic-only on doubles — 5.0.2 has no bitwise operators, no `%` and no `#`, and all three are parse errors there, so one occurrence stops the whole module loading whether the branch is taken or not. `mul32` splits its multiplier so no product exceeds 2^48; a direct `1812433253 * (2^32-1)` is 860× over 2^53 and would silently lose the low bits it carries forward.

Decisions are keyed — `{magic, iteration, purpose, index}` — not drawn from one running stream, so replaying a single cell does not depend on how many ran before it, including any that failed.

Instrument hazards

The generator's LAN service wedges, and there is no way to recover it remotely. No instrument on this bench has a smart plug, so every power cycle — a wedged SDG, the DMM's once-per-power-cycle UI build limit, a DMM control socket left held by a client that died — costs a human at the bench, and an unattended run that wedges is over until someone arrives. Two known triggers for the generator: a few consecutive large `WVDT` uploads, and sweeping SRATE across many rates on a very large stored waveform.

So a per-waveform health check asks both instruments whether they are still answering, using a Lua round trip rather than `*IDN?` — the SCPI parser can keep answering after the app is gone. On a silent instrument `bench_matrix` exits **3**, and `soak.py` ends the whole run on that rather than tallying laps against a dead bench. Exit 1 means "a cell failed" and a soak should carry on from it.

One client each. The DMM6500 accepts one controlling socket and the SDG one SCPI session; a second connection silently steals replies. `sdg_alive` must be passed an already-open handle or it reports a wedge that is not there.

Do not start a soak straight after a smoke — power cycle the DMM in between. `run_bench.py --start` now REFUSES to, so this is enforced rather than advised. `bench_smoke.py` sets a Lua global on the instrument before its first stage; `refuse_smoke_poison()` reads it and refuses while it is set. A power cycle clears it and nothing else sets it, so the condition is exactly "has a smoke run since the last power cycle". `--after-cycle` overrides it without even asking the instrument.

The mark is what makes it right in both directions, and a probe of `type(sdec)` was wrong in both: that asks "has anything loaded the app", so it **refused a resumed soak on a freshly cycled instrument** — `--start` sets `sdec` itself, so the second one always saw a table — and it **allowed a soak after a FAILED smoke**, because the receipt is only written on the pass path while the poison comes from running the

smoke rather than passing it. The mark is set before the first stage for that reason. An unreadable probe fails open and says so. `tools/test_pushplan.py` gates the decision table, including both of those cases and the fail-open branch.

It is enforced because it was advised three times and happened three times, the last one costing a whole detached run: **705 cells, every one idle.** This note was read on the evening of that occurrence, for another purpose, and the soak was started after a smoke anyway.

`acq: line is idle (no transitions)` is accumulated **DMM-side** state, and the controlled pairing is unambiguous: an identical 88-cell spec gave 63 idle of 88 cells with a stale DMM and a fresh SDG, and **0 of 88** with both fresh, the generator untouched between the halves. A soak started 30 minutes after a passing smoke went **33 idle of its first 49 cells**; after a DMM power cycle the same plan ran **0 idle in 6708 cells over 8.3 h**. `bench_smoke`'s `levels` stage sweeps the DMM down to 0.25 V of logic swing and its `rates` stage stubs `sdec.ua_badfrac` on the instrument, and `sdec.hw_config()` does not reset whatever this is.

It presents as a generator fault and is not one. The DMM reports a flat line while the generator reads back on the correct waveform with per-cell amplitude changes landing, and the record shows `SDG failed 0` because the engine's own generator checks never fire. Power-cycling the SDG changes nothing. **The cheap tell is cell time:** an idle cell fails in ~1 s against a healthy 5.4–6.4 s, so a poisoned run visibly races. It is not uptime either — the DMM has run a 7-day soak clean, so do not power cycle on a clock.

Watching a long run

`soak.py` captures each lap's output rather than streaming it, so a 155-minute lap is otherwise opaque from outside. `bench_matrix --heartbeat FILE` appends a flushed, timestamped line as each waveform starts and finishes. A healthy run appends one every ~2.5 min:

```
2026-08-26T05:46:18 iter 1 DONE  32/41 v51 43 cells 246s 5.72s/cell 0 badcells 0 inconc
0 baud 0 events DMM=alive SDG=alive
```

Read the fields as:

field	meaning
`32/41 v51`	waveform 32 of 41 this lap, and which one
`43 cells`	rate cells swept on this waveform
`0 badcells`	cells that did **not** decode as expected. Named to match the console's "not as expected"; double digits are normal for an impairment waveform, tagged `(loud, …)`, and for `v96`, which has an open issue
`0 inconc`	cells the judge declined — neither a pass nor a defect
`0 baud`	cells that misreported their rate
`0 events`	unexpected **instrument** events on this waveform. **This is the one that must be zero.** A `2205` or `2208` here means the app is filing file-system errors, which also pops them up on the panel
`DMM= SDG=`	whether each instrument still answered at the end of the waveform

`badcells` and `events` are different measurements and are easy to confuse: `events` counts the `*** unexpected event NNNN ***` lines, and those otherwise stay invisible until the lap ends, since the child's stdout is captured rather than streamed.

Do **not** use the child's CPU time as a liveness signal. The work is I/O bound at about 0.6 s of CPU an hour, and the child's pid changes at every lap boundary with its counter resetting to zero — so a monotonic check reports a stall at precisely the moment a lap completes. Judge liveness by the waveform number advancing and by the `DMM=` / `SDG=` footer: an `*IDN?` reply is not evidence, because the generator answers it with its waveform service wedged.

Every lap's per-cell output is written to `<record dir>/lap<n>-<verdict>.log` — including a lap the bench died under, tagged `STOPPED`. A lap that is not tallied is still examinable, which is what makes a later `grep` for an event code mean something.

The `run_bench.py` soak's own budget, and why it is not the 6.5 s above

`run_bench.py --start` is a different path from `soak.py`, with a different lap and a different cell time. Measured over 1607 and 116 cells with `--timing`, on the standard plan with all four diagnostics on:

phase	mean	note
`select+wait`	0.377 s	the generator conversation and the plan's wait
`acquire`	3.293 s	median 2.883, **p90 4.810** — the spread is the whole story
`decode`	1.736 s	
`record+panel`	0.010 s	0.2 % of the cell; the panel is not worth throttling
cell	**5.42–5.55 s**	phase sum, and wall clock over a full lap, agreeing

The cell time belongs to the PLAN, not to the harness. The same engine measured **4.945 s** on a plan built by `mkplan_rescale.py`, because that plan carries no low-baud cells — and it is the low-baud cells where `dmm.digitize.read` blocks longest in firmware, which is what the p90 of 4.81 s against a median of 2.88 s is showing. Quote the figure for the plan you are about to run, or do not quote one.

Do not pass `--skip-vectors` to `run_bench.py` at all

Its default is already `soakplan.HW_SKIP`, which is `v95,v96,v97`. Naming only the first two overrides that and re-admits v97, and the comment above `HW_SKIP` sets out exactly what that costs. Measured on two runs made that way, because this document's own next-step once spelled out the two-name form:

- the lap grows by one vector's worth of cells, 43 of them. With today's suite the correct skip gives 40 × 43 = **1720** and the two-name form gives 41 × 43 = **1763**; this list previously named 1720 as the misconfiguration signature, which it is not — 1720 is what the correct skip produces
- the skip is applied *before* the shuffle, so all 39 survivors take another vector's amplitude, offset and wait — the per-cell record stops being comparable with the archive, which is what a seeded plan is for. Aggregate rates survive; anything paired or keyed on (vector, rate, amplitude) does not
- **v97 then fails 100 % of its 43 cells, in every lap**, as one contiguous burst reported `SDG:ARWV? still names …` against whichever waveform preceded it in the draw — a different one each lap. That reads convincingly like a generator wedge, and it is a configuration error

A lap-size disagreement between the offline twin and the instrument is a bug, not a property of the two harnesses. Both should say whatever the planner says, which is the only authority worth quoting.

Deriving the lap is still better than assuming it: `run_bench.py` waits `60 + 12 × plan_rows` seconds, so the "waiting up to N min" line at startup gives the row count for *all* laps. One run cannot separate the header lines from the lap; two can. 1 lap gave 1725 rows and 4 gave 6885, so `h + d = 1725` and `h + 4d = 6885` give **d = 1720** and 5 header lines — which is the planner's figure, and an independent confirmation of it that sat two lines below the claim that a lap was 1677.

The instrument's clock is hours off the host's — ~4 h behind as measured. A record's progress rows are stamped with the DMM's own `os.time()`, so a difference between two record stamps is sound (that is where 5.55 s/cell comes from) and subtracting a host time from a record stamp is not.

Two fields on the panel exist for an unattended run. `spin N` on the note line is the count of spinner redraws in the cell just finished, which distinguishes a wedged run from a coalesced display: measured 10 redraws a cell arriving as 2 visible glyph steps, because the LCD only repaints where the interpreter yields. `mem NNNk` is the heap after the last collection — flat across lap boundaries is the answer you want, and a figure that only climbs is the one failure the cell counters cannot show.

Regrabbing the manual's figures

`tools/doc_shots.py` drives all nine panel screenshots in one pass and writes them into `docs/img`, which is version-controlled — so a bad grab replaces a shipped figure and the only way back is `git checkout`. Each shot therefore lands through a staging file renamed into place, and each is gated on three separate things:

check	asks	why it is not the others
`acted`	did the handler actually run	`ds_call` pcalls it and prints `ok=`. A refused press still leaves a screen to photograph.
`decoded`	did the capture produce bytes	only the frame shots consult a capture *shape*; a recording that got nothing has none.
`paged_ok`	is there really a page 2	`ui_page` is 0-based, so `page 0 of 1` passes any check that only reads the count.

A grab succeeding says the screen was read — not that anything was on it, and not that the press did anything. `--selftest` runs all three against nine recorded field sets with no instrument attached. `--only NAME[,NAME]` regrabs a subset, and `--retries` bounds the stochastic shapes: a mid-byte start is about one capture in eight, so the head shots retry.

A recording shot needs a locked rate, and clearing the lock is what makes the 32 kB shots come back empty. **Rewrite the captions from the new images rather than carrying them over** — a caption is a claim about a picture, and a stale one survives because nobody re-reads the figure. The trap when writing them: the paged figures are 32 kB recordings showing their retained **8 192-byte tail** (`sdec.ck_keep`), so the page counts are the tail's arithmetic, not the run's, and neither the byte count nor the page count can be read off the other.

What the instruments actually do, measured

Four things were measured directly rather than assumed, on 2026-09-07/08.

The generator's output envelope clamps the OFFSET. `|OFST| + AMP/2` cannot exceed 10 V, and when a commanded pair breaks that the SDG2122X keeps the amplitude and pulls the offset in. Its own `C1:BSWV?` reports the applied pair, so no scope is needed:

commanded	applied AMP	applied OFST	HLEV	LLEV
AMP 17.0, OFST +8.5	17.0	1.5	10.0	−7.0
AMP 10.0, OFST +6.0	10.0	5.0	10.0	0.0
AMP 4.0, OFST +12.0	4.0	8.0	10.0	6.0
AMP 20.0, OFST +3.0	20.0	0.0	10.0	−10.0

`OFST` lands on exactly `10 − AMP/2` every time and `HLEV` pins at 10.0 V, which confirms `soakplan`'s "at 20 Vpp the only legal offset is 0". `GEN_ENVELOPE` in `tools/gen_serial.lua` reproduces all four to 1e-6. `C1:OUTP?` reads `LOAD,HZ`, so commanded amplitudes reach the wire as commanded.

Reconstruction is NOT linear; `sweep_plan` interpolates linearly only because that fits the bench better than a hold. The generator is measured to hold: one edge captured at 1 GSa/s resolves a flat intermediate plateau held for one arb period to 0.04 %, with ~35 ns amplifier slews either side of it, and `RISE = FALL = 8.02 µs` at 125 kSa/s is the 10–90 % time across that whole staircase. See *The offline twin* above; this entry previously asserted linear as measured fact.

`v47`'s refusals are an edge-timing failure, not a level failure. `SER_Hello_8N1_Spike_x100` carries spikes in **both** directions, symmetric about the data: with the data at 1.88 V / 3.84 V, they reach 0.059 V and 5.696 V, i.e. 1.82 V below the space and 1.86 V above the mark. Because they are symmetric a min/max threshold is barely disturbed — 2.878 V against a true midpoint of 2.860 V — so the decision level is *not* corrupted. What breaks is timing: each spike is **two codewords wide and flat-topped** — `10813, 10813, 20643, 20643, 10813, 10813` in `v47.bin`, 5 of 5 — so it spans **16 µs at 125 kSa/s under any reconstruction**, and every crossing makes a spurious edge **pair**. At the cell's 100 µs sampling that is one sample wide, so `sig_bittime` floors at one sample and the app refuses with `0.9 samples/bit -- 11250 baud needs a faster capture`. Measured apex-hit rate: about **6 % of spikes**.

A scope window is not the waveform. At 100 µs/div the screen holds 1 ms of a 155 ms loop, and its honest `MAX 3.92 V` describes that 0.6 %. **Capture more than one loop period before describing a looping waveform** — a bare TSP digitize over 0.2 s at 1 MSa/s is what found the upward spike.

Soak results

Hardware, `out/bench/soak44`, 7 laps × 1677 = 11 739 cells over 17.3 h. 143 BAD, **1.22 %** (≤115200 Bd: 1.34 % [1.10–1.60]; >115200 Bd: 0.89 % [0.59–1.27]). **0 SDG failures**, against 129 in the previous run where a mis-specified skip list re-admitted `v97` — see the `--skip-vectors` warning above. 0 cells unrecorded, 0 events posted.

`acq: line is idle` was **0 in every one of the seven laps**. An earlier run degraded to 71.6 % idle at 12.5 h of DMM uptime, but two full `bench_smoke` runs had executed inside that uptime; 17.3 h clean with none rules out uptime alone. Heap at the six lap seams and the run total: 2678, 2678, 2679, 2678, 2679, 2678, 2678 kB — **flat to 1 kB over 10 062 cells**, so nothing leaks. Read the seam notes, not the panel's `mem`, which shows the *previous cell*'s working set and varies by ±100 kB with its rate.

One trend did appear, in a counter nobody was watching: **`v47`'s refusals rose 7, 7, 7, 9, 9, 11, 14 across the seven laps** — +1.11/lap, 5.1 σ, monotone, while `v48a` and `v48b` stayed flat. Mean commanded amplitude (correlation +0.43) and mean samples-per-bit (+0.22) do not explain it; lap number does (+0.92).

The 6 % apex-hit rate above is the quantity that would have to drift. Unconfirmed on one run.

Hardware, `out/bench/soak45_20260909.csv`, 4 laps × 1677 = 6708 cells over 8.3 h. 76 BAD, **1.13 %** (≤115200 Bd: 1.28 % [0.98–1.64]; >115200 Bd: 0.75 % [0.41–1.25]), and **0 no-decode, 0 inconclusive, 0 SDG failures, 0 events instigated, 0 s held over 0 retries, 0 cells unrecorded**, ending `4 iteration(s) complete`. Heap at the three lap seams: 2687, 2687, 2687 kB — **flat to 0 kB over 5031 cells**. Pooled with `soak44` the bench rate is **219 BAD of 18 447 = 1.19 %**, and the two runs agree within 0.6 σ.

`acq: line is idle` was **0 in every lap** — but the first attempt at this run, started 30 minutes after a passing smoke, went 33 idle in its first 49 cells and had to be abandoned. See the smoke warning under *Instrument hazards*: 8.3 h clean after a power cycle is further evidence against uptime.

`v47`'s refusal trend did not reproduce: 9, 9, 6, 8 across the four laps, flat and non-monotone. Every refusal in the run is on one of three vectors — `v48b` 168, `v48a` 158, `v47` 32 — and nothing else refuses at all. Four laps is short against a claimed +1.11/lap, so this neither confirms nor kills the `soak44` trend; it does mean it is not reliably present.

The late ARWV switch reproduced at exactly its documented rate. 16 recoveries over 4 laps — 4 a lap against 39 real switches, **10.3 %** — every one logged `sdg select recovered on try 2` or `try 3` after the 4 in-call `ARWV?` retries had all returned the previous waveform's name. Re-sending the *selection* is what recovers it; re-reading is what does not.

Offline, two pools of `tools/soak_offline_bench.py` under Lua 5.0.2, 6 h. 5068 laps, **8.50 M cells** — comparable to the entire prior offline history, and the first with a varying capture phase. Zero `raised`, zero crashes, zero lost records, zero `nobytes`. The pools differed in one variable: half the cells in the second commanded an out-of-envelope pair, so 838 were recentred a lap and 517 straddled ground. Paired by seed over 742 laps the cost was **+4.635 fails/lap, 95 % CI +4.53 to +4.74, higher in 742 of 742 pairs** — about 0.9 % of the cells affected. The archive's 9.55× byte-failure ratio on recentred cells implies ~49, so something outside the envelope model accounts for the rest; the DMM front end is the largest path still unmodelled.